

Reviewer Pack — Minimal Rank of Quasi-Group Representations and Circuit Weig...

Entry caffc262c8b4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 12:12:10 UTC

Minimal Rank of Quasi-Group Representations and Circuit Weights Correlation

Entry ID: caffc262c8b4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 12:12:10 UTC

1. Conjecture

Field A (mathematical branch): Quasi-group Theory **Field B** (complexity object): Boolean Circuits

Statement:

For every CNF φ with n variables, the minimal rank of a noncommutative representation of the quasi-group associated with φ 's boolean algebra is linearly correlated with its circuit weight $w(\varphi)$, such that $\min_rank(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Quasi-groups provide an algebraic framework to study symmetry in non-associative operations, which can be related to the structure of circuits. By analyzing the minimal rank of representations, we might uncover structural properties of circuits that are not apparent through standard methods.

Taxonomy category: Quasi_group_circuit_weights (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 57f22762fa1719f3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\min_rank(\varphi)$ and $w(\varphi)$ for all CNFs φ with n variables ($n \leq 40$) is greater than or equal to 0.8, AND the mean of the correlation coefficients across at least 30 seeds is also greater than or equal to 0.8.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" AND "quasi-group representations" AND Boolean circuits"
- "CNF" AND boolean algebra AND circuit weight" - "noncommutative representation" AND quasi-group theory AND correlation with circuit complexity

Top relevant hits considered: - [http://arxiv.org/abs/1403.6863v1] Online Learning of k-CNF Boolean Functions - [http://arxiv.org/abs/2112.06062v1] A Critique of Kumar’s “Necessary and Sufficient Condition for Satisfiability of a Boolean Formula in CNF and Its Implications” - [http://arxiv.org/abs/2504.06476v2] Accelerating Hybrid XOR–CNF Boolean Satisfiability Problems Natively with In-Memory Computing

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n - 1):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if sum(clause) == 0:
                continue
            clauses.append(clause)
        return clauses

    def boolean_algebra_quasi_group(cnf):
        n = len(cnf[0])
        quasi_group = {}
        for clause in cnf:
            for x in range(1 << n):
                if all((x & (1 << abs(l) - 1)) == l * sign for l, sign in enumerate(clause)):
                    for y in range(1 << n):
                        if all((y & (1 << abs(l) - 1)) == l * sign for l, sign in enumerate(clause)):
                            result = x ^ y
                            if result not in quasi_group:
                                quasi_group[result] = set()
                            quasi_group[result].add(x)
                            quasi_group[result].add(y)
```

```

    return quasi_group

def min_rank(quasi_group):
    n = len(quasi_group)
    adjacency_matrix = [[0] * n for _ in range(n)]
    for i, x in enumerate(quasi_group):
        for y in quasi_group:
            if y in quasi_group[x]:
                adjacency_matrix[i][quasi_group.index(y)] = 1

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for i in range(cols):
        pivot_row = -1
        for j in range(rank, rows):
            if matrix[j][i] != 0:
                pivot_row = j
                break
        if pivot_row == -1:
            continue

        matrix[pivot_row], matrix[rank] = matrix[rank], matrix[pivot_row]
        rank += 1

        for j in range(rows):
            if j != rank - 1:
                factor = matrix[j][i] / matrix[rank - 1][i]
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[rank - 1][k]

    return rank

return gaussian_elimination(adjacency_matrix)

def circuit_weight(cnf):
    return len(cnf) + sum(len(clause) - 1 for clause in cnf if len(set(abs(l) for l in clause)) > 1)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    cnf = generate_cnf(n)
    quasi_group = boolean_algebra_quasi_group(cnf)
    min_rank_value = min_rank(quasi_group)
    circuit_weight_value = circuit_weight(cnf)
    results.append((min_rank_value, circuit_weight_value))

if len(results) < 30:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }
}

```

```

min_rank_values, circuit_weight_values = zip(*results)
mean_min_rank = sum(min_rank_values) / len(min_rank_values)
mean_circuit_weight = sum(circuit_weight_values) / len(circuit_weight_values)
correlation_coefficient = (sum((x - mean_min_rank) * (y - mean_circuit_weight) for x, y in zip
                             math.sqrt(sum((x - mean_min_rank)**2 for x in min_rank_values) *
                             sum((y - mean_circuit_weight)**2 for y in circuit_weight_

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fra
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_less_than_0.8\" first_f

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba2111d7.py", line 122, in <module>
    trial_result = run_trial(seed)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba2111d7.py", line 85, in run_trial
    quasi_group = boolean_algebra_quasi_group(cnf)
    ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba2111d7.py", line 35, in boolean_alge
    if all((x & (1 << abs(l) - 1)) == l * sign for l, sign in enumerate(clause)):
    ~~~~~

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ba2111d7.py", line 35, in <genexpr>
    if all((x & (1 << abs(l) - 1)) == l * sign for l, sign in enumerate clause)):
    ~^^~
```

ValueError: negative shift count

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to a ValueError, which prevented the production of data necessary to evaluate the conjecture. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14436
2	preregistration	ollama_remote	glm4:latest	0	0	13176
3	novelty	ollama_remote	glm4:latest	0	0	8355
4	novelty	ollama_remote	glm4:latest	0	0	13533
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14036
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51428
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	46597
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	157000
9	judge	ollama_remote	glm4:latest	0	0	43629

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 362191 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/caffc262c8b4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/caffc262c8b4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/caffc262c8b4.tar.gz` (if generated)